

INFO 0702 CM3

Conteneurs système

Emilien Bourdy-Liébart



**UNIVERSITÉ
DE REIMS
CHAMPAGNE-ARDENNE**

10 octobre 2025

Sommaire

1 LXC

2 cgroups

Généralités I

- *Linux containers*
- Licence LGPLv2.1+
- Isolation au niveau système d'exploitation
- Basé sur **cgroups**
- Partage
 - du noyau avec l'hôte
 - d'une partie du disque

Généralités II

- Virtualisation de l'environnement d'exécution
 - mémoire
 - hiérarchie de fichier
 - processeur
 - réseau
- `https://linuxcontainers.org`
- Quelque part entre un `chroot` et une machine virtuelle complète

Généralités III

Features

- *namespaces*
 - IPC
 - UTS
 - *mount*
 - PID
 - réseau
 - *user*
- Apparmor et SELinux → sécurité
- Seccomp → changement d'état unidirectionnel
- chroot
- *Kernel capabilities*
- cgroups

Mise en œuvre

- Installation sans modification du noyau
- Installation
 - `lxc`
 - `lxcctl`
- Vérification
 - `lxc-checkconfig`

Commandes de base I

Création

- `lxc-create`

Exemple (Commandes de base (1/4))

```
root:~# lxc-create --name mycontainer \  
--template download  
# Sélection de la distribution à installer  
# Ou lxc-create --name mycontainer \  
--template download -- --dist alpine --  
release 3.22 --arch amd64  
root:~# ls /var/lib/lxc  
mycontainer  
root:~# ls /var/lib/lxc/mycontainer  
config    rootfs
```

Commandes de base II

Utilisation

- `lxc-start` (-F en mode interactif)

Exemple (Commandes de base (2/4))

```
root:~# lxc-start --name mycontainer
root:~# lxc-info --name mycontainer
Name:          mycontainer
State:         RUNNING
PID:           5107
IP:            10.0.3.38
Link:          vethbFFAck
TX bytes:      1.80 KiB
RX bytes:      2.49 KiB
Total bytes:   4.29 KiB
```


Commandes de base III

Rentrer dans le container

- `lxc-attach`

Exemple (Commandes de base (3/4))

```
root:~# lxc-attach --name mycontainer
/ # /bin/cat /etc/os-release
NAME="Alpine Linux"
ID=alpine
VERSION_ID=3.22.1
PRETTY_NAME="Alpine Linux v3.22"
HOME_URL="https://alpinelinux.org/"
BUG_REPORT_URL="https://gitlab.alpinelinux.org/alpine/
aports/-/issues"
/ # exit
```

Commandes de base IV

Arrêter un container

- `lxc-stop`
- `lxc-destroy`

Exemple (Commandes de base (4/4))

```
root~#: lxc-stop --name mycontainer
root:~# lxc-info --name mycontainer
Name:          mycontainer
State:         STOPPED
root:~# lxc-destroy --name mycontainer
root:~# ls /var/lib/lxc
[mycontainer absent]
```

Autres commandes

- Démarrage automatique

```
root:~# echo "lxc.start.auto = 1" >> /var/lib/lxc/mycontainer/config
```

- Démarrage automatique de plusieurs container

```
root:~# cp /etc/lxc/default.conf /etc/lxc/autostart.conf
root:~# echo "lxc.start.auto = 1" >> /etc/lxc/autostart.conf
root:~# lxc-create --name ... --config /etc/lxc/autostart.conf
```

- Montage de volume

```
root:~# lxc-attach --name mycontainer -- mkdir -p \
/container/mount/point
root:~# echo "lxc.mount.entry = /host/path/to/volume \
container/mount/point none bind 0 0" \
>> /var/lib/lxc/mycontainer/config
```

- Cloner un container

```
root:~# lxc-copy -n base -N snap1 -B overlayfs -s
```

Utiliser LXC sans être root I

❶ Générer des containers avec UID/GID séparés

Premier container :

```
root~#: echo "root:100000:65536" >> /etc/subuid
root~#: echo "root:100000:65536" >> /etc/subgid
root~#: cp /etc/lxc/default.conf /etc/lxc/container1.conf
root~#: echo "lxc.idmap = u 0 100000 65536" >> /etc/lxc/container1.conf
root~#: echo "lxc.idmap = g 0 100000 65536" >> /etc/lxc/container1.conf
root~#: lxc-create --config /etc/lxc/container1.conf --name container1
--template download
```

Second container :

```
root~#: echo "root:200000:65536" >> /etc/subuid
root~#: echo "root:200000:65536" >> /etc/subgid
root~#: cp /etc/lxc/default.conf /etc/lxc/container1.conf
root~#: echo "lxc.idmap = u 0 200000 65536" >> /etc/lxc/container2.conf
root~#: echo "lxc.idmap = g 0 200000 65536" >> /etc/lxc/container2.conf
root~#: lxc-create --config /etc/lxc/container2.conf --name container2
--template download
```

Utiliser LXC sans être root II

- ② Ajouter les droits à l'utilisateur de créer des interfaces réseau
- ③ Créer une configuration par utilisateur
- ④ Copier `/etc/lxc/default.conf` dans le répertoire de configuration de l'utilisateur
- ⑤ Ajouter les `lxc.idmap` dans la configuration

```
user:~$ echo "$ (id -un) veth lxcbr0 10" | sudo tee -a \
/etc/lxc/lxc-usernet
user:~$ mkdir -p ~/.config/lxc
user:~$ cp /etc/lxc/default.conf ~/.config/lxc/default.conf
user:~$ MS_UID="$(grep "$(id -un)" /etc/subuid | cut -d : -f 2)"
user:~$ ME_UID="$(grep "$(id -un)" /etc/subuid | cut -d : -f 3)"
user:~$ MS_GID="$(grep "$(id -un)" /etc/subgid | cut -d : -f 2)"
user:~$ ME_GID="$(grep "$(id -un)" /etc/subgid | cut -d : -f 3)"
user:~$ echo "lxc.idmap = u 0 $MS_UID $ME_UID" >> \
~/.config/lxc/default.conf
```

Utiliser LXC sans être root III

```
user:~$ echo "lxc.idmap = g 0 $MS_GID $ME_GID" >> \  
~/.config/lxc/default.conf
```

⑥ Sur Ubuntu LTS 20.04, rajouter

```
export DOWNLOAD_KEYSERVER="hkp://keyserver.ubuntu.com"
```

Pour créer un container :

```
user:~$ systemd-run --unit=my-unit --user --scope -p "Delegate=yes" \  
-- lxc-create --name mycontainer --template download  
user:~$ systemd-run --unit=my-unit --user --scope -p "Delegate=yes" \  
-- lxc-start --name mycontainer
```

Erreur au lancement

- Sur certaines distributions, `lxc-start` n'arrive pas à faire `lxcbr0`
- Installer `dnsmasq`
- Ajouter un fichier `/etc/default/lxc-net` avec :
USE_LXC_BRIDGE="true"
LXC_BRIDGE="lxcbr0"
LXC_ADDR="10.0.3.1"
LXC_NETMASK="255.255.255.0"
LXC_NETWORK="10.0.3.0/24"
LXC_DHCP_RANGE="10.0.3.2,10.0.3.254"
LXC_DHCP_MAX="253"
LXC_DHCP_CONFILE=""
LXC_DOMAIN=""
- Redémarrer le service `lxc-net`

Questions

Exercice (lxc)

- ❶ Quelle est la commande pour créer un container ?
- ❷ Où sont placés les containers ?
- ❸ Comment exécuter un container en mode interactif ?
- ❹ Quelle est la commande pour avoir des informations sur un container ?
- ❺ Comment entrer dans un container lancé en tâche de fond ?
- ❻ Comment arrêter un container ?
- ❼ Comment détruire un container ?
- ❽ Comment cloner un container ?

Qu'est-ce que c'est ?

- Gestion des ressources systèmes attribuées aux processus
- permet de les limiter
- CPU
 - mémoire
 - bande passante
 - etc.
- Noyau Linux 2007 (← développeurs Google)
 - Chaque groupe de ressource (sous-système) indépendant
 - Depuis **cgroups** v2, unification de la gestion des ressources en un seul espace

Structure d'un cgroup

- `/sys/fs/cgroup`
- `cpu.max` → limite de temps CPU par groupe de processus
- `memory.max`, `memory.current` → gestion mémoire
- `io.max` → entrée/sortie périphériques
- bande passante (encore en développement au 23/05/2025)

Création d'un cgroup

- Créer un cgroup
- création d'un sous-répertoire dans `/sys/fs/cgroup`
- Indiquer dans `cgroup.subtree_control` les contrôleurs utilisés
 - Ajouter dans `cgroup.procs` les processus concernés
 - Ajouter un fichier pour chaque limite de ressource (`memory.max`, `cpu.max`, etc.)

Exemple I

```
# Création du répertoire
root:~# mkdir /sys/fs/cgroup/mon_cgroup
# Ajout des contrôleurs
root:~# echo "+cpu +memory" > /sys/fs/cgroup/mon_cgroup/
    cgroup.subtree_control
# Ajout des pids
root:~# echo 1234 > /sys/fs/cgroup/mon_cgroup/cgroup.procs
# 50 % du cpu
root:~# echo "50000 100000" > /sys/fs/cgroup/mon_cgroup/
    cpu.max
```

Exemple II

```
# 512 Mo
root:~# echo "5368709122" > /sys/fs/cgroup/mon_cgroup/
      memory.max
# Lecture limitée à 1 Mo sur un périphérique
root:~# echo "8:0 rbps=10248576" > /sys/fs/cgroup/
      mon_cgroup/io.max
# Vérification
root:~# cat /sys/fs/cgroup/mon_cgroup/memory.current
```

Avec systemd

- systemd génère un cgroup pour chaque service
- `/sys/fs/cgroup/system.slice/<service>.service/`
- Configuration dans le fichier `.service` associé

Exemple I

```
# Démarrage du service
root:~# systemctl start nginx
root:~# ls /sys/fs/cgroup/system.slice/ | grep nginx
nginx.service
root:~# ls /sys/fs/cgroup/system.slice/nginx.service |
      grep cpu
cpu.pressure
cpu.stat
cpu.stat.local
# Modification du fichier /usr/lib/systemd/system/
# nginx.service
[Service]
CPUQuota=50%
MemoryMax=512M
```

Exemple II

```
# Redémarrage du service
root:~# systemctl daemon-reload
root:~# systemctl restart nginx
root:~# ls /sys/fs/cgroup/system.slice/nginx.service |
        grep cpu
cpu.idle
cpu.max
cpu.max.burst
cpu.pressure
cpu.stat
cpu.stat.local
cpu.uclamp.max
cpu.uclamp.min
cpu.weight
cpu.weight.nice
```


Exemple III

```
root:~# cat /sys/fs/cgroup/system.slice/nginx.service/  
        cpu.max  
50000 100000
```

Obtenir des informations sur les cgroups

- `systemctl status`
 - max, disponible etc.
- `systemd-cgtop`
 - comme `top`, mais pour les cgroup utilisés par `systemd`
- `systemd-cgls` dans `system.slice`
 - affichage de tous les cgroup gérés et comment

Questions

Exercice (cgroups)

- ❶ Dans quel répertoire se situent les `cgroups` ?
- ❷ Comment créer un `cgroup` ?
 - ❶ Création ?
 - ❷ Contrôleurs ?
 - ❸ Processus ?
 - ❹ Limite de ressources ?
- ❸ Dans quel répertoire `systemd` met-il ses `cgroups` ?
- ❹ Comment limiter des ressources avec `systemd` ?
- ❺ Comment afficher le statut des `cgroups` ?
- ❻ Comment afficher les `cgroups` en cours d'utilisation ?
- ❼ Comment afficher les `cgroups` existant ?